Fine-grained ML SATD Classification with Large Language Models: An Empirical Study

ANONYMOUS AUTHOR(S)

Identifying self-admitted technical debt (SATD), which developers explicitly document in source-code comments, is an important step toward managing software quality. Although SATD has been widely studied in traditional software systems, machine learning (ML) software introduces additional maintenance concerns that are related to data processing, model configuration, dependencies, reliability, and prediction quality. These concerns make ML SATD more fine-grained and difficult to classify than traditional SATD. Existing studies have mainly focused on manually constructing taxonomies and qualitatively characterizing ML-specific SATD. However, it remains unclear whether such fine-grained debt categories can be automatically classified in low-resource settings. This paper presents an empirical study of large language models (LLMs) for fine-grained ML-SATD classification. Using a processed version of the OBrien dataset with 23 ML-specific SATD categories, we evaluate nine LLMs under different prompting strategies, including direct label selection, binary decomposition, hierarchical prompting, LLM-generated examples, and reasoning-oriented prompts. Our results show that LLMs provide a promising solution for this low-resource classification task. Vanilla and binary prompting consistently outperform hierarchical prompting, suggesting that multi-stage label decomposition may introduce error propagation. A small number of generated examples can improve performance, whereas adding more examples does not necessarily lead to further gains. Reasoning-oriented prompting further improves classification effectiveness. By combining these strategies, the best LLMs Qwen-8B achieve a weighted F1-score of 0.381. Our findings provide practical guidance for applying LLMs to ML-SATD classification and a baseline for this area, reveal remaining challenges caused by label imbalance, semantic overlap, and rare debt categories.

CCS Concepts: • **Software and its engineering → Maintaining software**.

Additional Key Words and Phrases: Self-Admitted Technical Debt, Large Language Models, Low-Resource Text Classification

ACM Reference Format:
Anonymous Author(s). 2018. Fine-grained ML SATD Classification with Large Language Models: An Empirical Study. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 26 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

In recent years, the rapid advances and widespread adoption of artificial intelligence (AI), especially machine learning (ML) and deep learning (DL), have enabled software systems to address problems that were previously difficult or even infeasible with traditional techniques. ML-enabled systems have been increasingly deployed in various domains, including medical diagnosis [], financial fraud detection [], autonomous driving [], and software engineering [**?**]. However, while ML components provide powerful capabilities, their fast integration into long-lived software systems also introduces substantial maintenance challenges [26].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Self-admitted technical debt (SATD) provides a practical way to observe such maintenance concerns. SATD refers to debt that developers explicitly document in source-code comments, such as temporary workarounds, incomplete implementations, fragile assumptions, or code that should be improved in the future []. This types of technical debt is also commonly appears in machine learning (ML) software. Figure 1 shows an example of ML-specific SATD from the open-source GitHub project `magpie`. In this example, the developer notes that the current output vector is overly sparse and suggests improving its memory efficiency in a future version. Such comments reflect not only acknowledged design or implementation concerns, but also potential risks that may affect the maintainability and evolution of ML systems. Therefore, identifying the fine-grained types of ML-SATD comments can help developers better understand project risks and support more effective technical debt management.

In recent years, the rapid advances and widespread adoption of artificial intelligence (AI), especially machine learning (ML) and deep learning (DL), have enabled software systems to address problems that were previously difficult or even infeasible with traditional techniques. ML-enabled systems have been increasingly deployed in various domains, including medical diagnosis [], financial fraud detection [], autonomous driving [], and software engineering [**?**]. However, while ML components provide powerful capabilities, their fast integration into long-lived software systems also introduces substantial maintenance challenges [26].

Self-admitted technical debt (SATD) provides a practical way to observe such maintenance concerns. SATD refers to debt that developers explicitly document in source-code comments, such as temporary workarounds, incomplete implementations, fragile assumptions, or code that should be improved in the future []. This types of technical debt is also commonly appears in machine learning (ML) software. Figure 1 shows an example of ML-specific SATD from the open-source GitHub project `magpie`. In this example, the developer notes that the current output vector is overly sparse and suggests improving its memory efficiency in a future version. Such comments reflect not only acknowledged design or implementation concerns, but also potential risks that may affect the maintainability and evolution of ML systems. Therefore, identifying the fine-grained types of ML-SATD comments can help developers better understand project risks and support more effective technical debt management.

ML SATD is more complex than the traditional SATD.

[4] [] []

These technical debt contain all the traditional SATD and the ML-Specifical SATD []. ML software is more complex than traditional software due to that the software including AI components

Prior work has shown that ML-specific debt is often related to data processing, model configuration, dependencies, model interpretability, prediction quality, and reliability. To better characterize these issues, O'Brien et al. proposed a fine-grained taxonomy of SATD in ML software, covering 23 ML-specific debt categories. These categories capture maintenance concerns that are difficult to represent using conventional SATD taxonomies, such as machine learning knowledge, data processing configuration, model code reusability, prediction quality, and machine learning reliability.

Technical debt is a Existing studies have provided valuable taxonomies and empirical characterizations of ML-specific SATD, but they remain largely manual and descriptive. This limits their practical use in large and evolving ML repositories, where developers may produce many comments and where manually assigning fine-grained debt labels is time-consuming. More importantly, fine-grained ML-SATD classification is more challenging than conventional SATD detection. The task requires distinguishing among many semantically related debt categories, while the available labeled data are limited and highly imbalanced. These characteristics make it difficult to directly apply traditional supervised classifiers, which usually require sufficient labeled examples for each category.

Large language models (LLMs) offer a potential alternative for this low-resource classification setting. Since LLMs can follow natural-language instructions and perform classification through prompting, they may reduce the need for task-specific model training. However, it remains unclear whether LLMs can reliably classify fine-grained ML-SATD comments, and how well are they performanced. The difficulty lies not only in selecting the correct label from a large label set, but also in understanding subtle maintenance concerns expressed in short and informal developer comments. In addition, prompt design may substantially affect classification outcomes. A direct prompt may be simple but cognitively demanding; a hierarchical prompt may reduce the decision space but introduce error propagation; in-context examples may clarify label boundaries but may also add noise; and reasoning-oriented prompts may help models distinguish overlapping categories. To the best of our knowledge, there is no work focus on inversigate these aspect, These motivates our to conducted a systematic empirical investigation.

In this paper, we study how to using LLMs for fine-grained ML-SATD classification. LLMs can effectively classifying fine-grained ML-SATD without the need for task-specifical training. We first select a well constructed and wildly used dataset, OBrien et al. [16] constructed a Using a processed single-label subset of O'Brien et al.'s ML-SATD dataset, we evaluate nine LLMs across multiple prompting strategies. Specifically, we examine direct label selection, binary decomposition, hierarchical prompting, LLM-generated in-context examples, reasoning-guided prompting, and combinations of these strategies. Our goal is not only to measure the performance of LLMs on this task, but also to understand how prompt structure, examples, and reasoning affect their ability to distinguish ML-specific technical debt categories.

We address the following research questions:

- **RQ1:** How do different prompt structures affect LLM-based fine-grained ML-SATD classification?
- **RQ2:** To what extent do generated in-context examples help LLMs distinguish fine-grained ML-SATD categories?
- **RQ3:** How does reasoning guidance influence LLM performance on semantically overlapping ML-SATD labels?
- **RQ4:** Can the effective prompting strategies be combined to further improve LLM-based fine-grained ML-SATD classification?

Contemporary software systems have enpowered by AI conpornent, it enpowered lots of softwares and maintaining such systems become cretical. Although development these software are very quickly, maintaining these software remains challenging [23].

To trace and manage technical debt, prior research has largely focused on self-admitted technical debt (SATD) [18], a type of technical debt documented by developers in source code comments. Although SATD has been widely studied in traditional software systems, it also commonly appears in machine learning (ML) software. Figure 1 shows an example of ML-specific SATD from the open-source GitHub project `magpie`. In this example, the developer notes that the current output vector is overly sparse and suggests improving its memory efficiency in a future version. Such comments reflect not only acknowledged design or implementation concerns, but also potential risks that may affect the maintainability and evolution of ML systems. Therefore, identifying the fine-grained types of ML-SATD comments can help developers better understand project risks and support more effective technical debt management.

These studies regard the identification and classification of SATD as the first step toward its effective management. Most of them rely on learning-based methods to detect and classify SATD in traditional software systems. In general, these approaches can be summarized in two main steps:

```
214  294          # Create the output vector
     295  +       # TODO this vector is very sparse, we can make it more memory efficient
215  296          output_vector = []
216  297          for kw in kw_candidates:
217  298              if kw.get_canonical_form() in doc_answers:
218  299                  output_vector.append(1)  # True
219  300              else:
220  301                  output_vector.append(0)  # False
221  302
```

Fig. 1. Example of ML SATD in the open source project.

1) collecting a large training dataset, and 2) training a task-specific classifier on the dataset, which is then used to identify and classify SATD [6, 30].

However, when it comes to Self-admitted technical debt in machine learning (ML SATD) these approaches are failed. ML systems face all the maintenance problems of conventional code, plus an additional set of ML-specific issues. Making it more complex than the SATD taxonomy in traditional software [19]. There are two main obstracal using learning-based approach to classifiy fine-grained ML SATD. The first one is 1) The label of fine-grained ML SATD is more complex and spare. It involves specialized and more complex types than traditional SATD [19]. 2) The datasets are really small. It is hard to collect a lots of training datasets which always exceed the testing datasets size and fine-tuning a task specifical model for classification it. As a results, to the best of our knowledge, no work present an automatic method to classify ML SATD. Prior work has proposed taxonomies of SATD in AI-powered systems [5, 17] and conducted empirical analyses [2, 4] of the patterns of these SATD. These studies mainly focus on manually analyzing SATD types in AI-powered systems, they manually the detailed types and patterns can help identify ML SATD and facilitate reducing the maintenance cost of the software.

Large Language Models (LLMs) have the potential on tackling this problem by using prompting without task-specific training. This method sheild light on the fine-grained ML SATD classification, Some research have study using it to identificy the SATD in traditional software []. However their effectiveness on the fine-grained ML SATD classification task still need explored. We fill this gap in this study, we explored how to better using these LLMs on this task. Prompting is the most important thing in this we evaluate different prompting methods. To understanding how to better prompting these LLMs and know their performance on this task. We first explored three prompting methods,

By addressing these research questions, this study not only sheds light on the current capabilities and limitations of LLMs in the area of ML SATD classification but also provides actionable guidance for leveraging LLMs effectively in software engineering research and practice.

Our empirical study address four research questions:

RQ1: How to prompt LLMs for fine-grained ML SATD classification?

RQ2: Can incorporating LLM-generated examples improve the performance of LLMs on fine-grained ML SATD classification?

RQ3: Can reasoning improve the performance of LLMs on fine-grained ML SATD classification?

RQ4: Can combining these approaches further improve LLM performance in the fine-grained classification of ML SATD?

Summary of findings We evaluate nine LLMs to reveal how to , and the results show that vanilla and binary prompting are the two most effective prompting strategies for this task (RQ1). We also find that model-generated examples can improve performance, although adding more

examples does not necessarily lead to better results; in particular, using no more than three examples yields the best performance (RQ2). In addition, reasoning mode further improves the results, and incorporating manually written chain-of-thought examples helps the models perform better on the classification task (RQ3). By combining these strategies, the best-performing model is Qwen3-8B under the 3-shot setting, achieving an F1 score of 38.1% (RQ4).

Our contributions are fourfold:

(1) We conduct the first and largest empirical study on the performance of LLMs in fine-grained ML SATD classification. A total of nine LLMs, covering both general-purpose and code-oriented models, reasoning models, are evaluated to assess their effectiveness on this task.

(2) We explore different prompting strategies, including vanilla, binary, and hierarchical prompting. The results provide practical guidelines for selecting effective prompts for fine-grained ML SATD classification.

(3) We explored two different ways automatically generated examples and reasoning mode and demonstrate their effectiveness on this task.

(4) We further explore combinations of these prompting methods and report the best performance achieved by current LLMs on this task, which can serve as a strong baseline for future studies.

The rest of the paper is organized as follows. Section 2 introduces the LLMs and summarizes the current state of ML SATD classification. Section 3 gives the detailed datasets, models, parameters, and evaluation metrics used in this paper and the detailed methodology for each RQ. Section 4 presents the results of each RQ and summarizes the findings. Section 6 give a detailed discuss of the lessons learned from the experiments. Section 7 give the threats to validity. Section 8 concludes our paper.

2 RELATED WORK

Our work lies at the intersection of two lines of research: automated identification and classification of self-admitted technical debt (SATD), and technical debt in machine learning (ML) software. We review these two areas and position our study with respect to existing SATD classification techniques and ML-specific debt taxonomies.

2.1 Automated Identification and Classification of Self-Admitted Technical Debt

SATD refers to technical debt that developers explicitly acknowledge in software artifacts, particularly source-code comments [18]. Early studies investigated the identification and categorization of SATD in traditional software systems [14]. Subsequent work automated SATD detection using natural-language processing, text mining, and supervised learning techniques [7, 9, 30]. Neural models were later introduced to improve representation learning, cross-project prediction, and the interpretability of SATD detection [20]. Collectively, these studies demonstrated that textual information in developer comments can effectively support automated SATD analysis, although most early approaches primarily focused on distinguishing SATD from non-SATD.

Researchers have subsequently extended SATD analysis from binary detection to multi-class classification. Chen et al. [6] formulated SATD classification as a multi-class learning problem and investigated the use of XGBoost for distinguishing among general SATD categories. Deep-learning approaches have also been explored to improve multi-type SATD classification, particularly under imbalanced class distributions [32]. More recently, Arcelli Fontana et al. [3] systematically revisited binary and multi-class SATD classification, providing an updated assessment of the capabilities and limitations of existing classification techniques. These studies advance automated SATD analysis beyond binary detection, but their classification targets remain predominantly conventional software debt categories.

Large language models (LLMs) have recently provided an alternative to task-specific supervised classifiers. Sheikhaei et al. [24] systematically evaluated LLMs for SATD identification and classification under fine-tuning and in-context learning settings, and examined the effects of demonstrations, category descriptions, model scale, and contextual information. Their results demonstrate the potential of LLMs for SATD analysis and highlight the importance of prompt configuration. However, their classification experiments on the O'Brien dataset consider six general SATD categories—Requirement, Code, Test, Defect, Design, and Documentation—rather than the fine-grained ML-specific taxonomy. Other studies have explored hybrid approaches combining LLMs with smaller neural models [12], while Arcelli Fontana et al. [3] also evaluated LLMs in multi-class SATD classification. Therefore, although LLM-based SATD classification has received increasing attention, their effectiveness in distinguishing fine-grained ML-specific debt types remains insufficiently understood.

2.2 Technical Debt and SATD in Machine Learning Software

ML software introduces maintenance concerns that extend beyond those typically encountered in traditional software. Early studies highlighted forms of technical debt arising from data dependencies, configuration complexity, pipeline entanglement, hidden feedback loops, and interactions among ML components [5, 22, 23]. Subsequent empirical studies provided further evidence of domain-specific technical debt in ML and deep-learning systems [2, 13]. More recently, Bhatia et al. [4] investigated SATD in ML software at scale and showed that SATD is prevalent across ML development activities, further motivating dedicated techniques for managing technical debt in ML systems.

To characterize these domain-specific concerns, researchers have developed specialized SATD taxonomies for ML and deep-learning software. OBrien et al. [16] conducted an empirical study of SATD in ML repositories and introduced a fine-grained taxonomy consisting of 23 ML-specific SATD types. These categories capture concerns related to data, model configuration, dependencies, ML knowledge, model code, prediction quality, reliability, and other aspects of ML software that are not adequately represented by conventional SATD categories. Pepe et al. [17] further investigated SATD in deep-learning systems and proposed a DL-specific taxonomy, providing additional evidence that the technical-debt landscape of AI-enabled software contains domain-specific concerns requiring finer-grained abstractions.

Recent studies have begun to automate the analysis of such domain-specific technical debt. Simon et al. [25] investigated automated detection of Algorithm Debt in deep-learning frameworks using machine-learning, transformer-based, and LLM-based representations, demonstrating the feasibility of automatically identifying a specialized debt type. In a complementary direction, Nakashima et al. [15] investigated the use of LLMs to automate SATD taxonomy construction across multiple software domains, including ML software. These studies, however, address either the detection of a particular specialized debt type or the construction of domain-specific taxonomies. In contrast, systematically assigning SATD comments to the 23 fine-grained ML-specific categories established by O'Brien et al. [16] remains insufficiently explored. Our study bridges this gap by evaluating open-weight LLMs for fine-grained ML-SATD classification without task-specific training and by systematically investigating how prompt structure, synthetic demonstrations, and reasoning guidance affect their ability to distinguish semantically related ML-specific debt types.

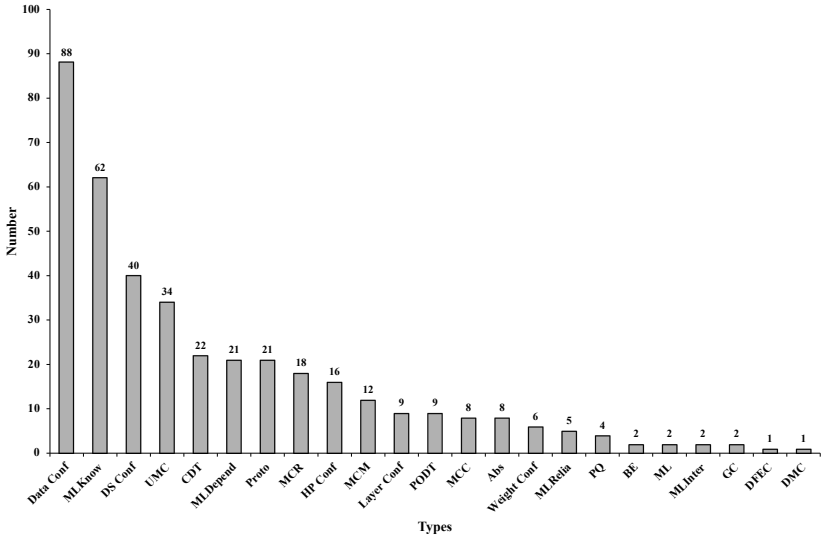


Fig. 2. Statistical info of the processed dataset.

3 METHODOLOGY

3.1 Datasets

O'Brien Dataset. We use the dataset released by O'Brien et al. [16], which contains 856 manually labeled SATD comments collected from Python-based machine learning (ML) repositories. The dataset annotates SATD along two dimensions: *Software SATD Types* for general software technical debt and *ML SATD Types* for ML-specific technical debt. The latter consists of 23 fine-grained categories. Since the original ML-specific annotations may be absent or multi-label, we preprocess the dataset to construct a single-label evaluation set for our classification task.

Data Processing. Starting from the 856 SATD comments, we first exclude 377 comments without an ML-specific annotation, leaving 479 comments with at least one ML-specific label. We then remove 77 comments annotated with multiple ML-specific types, resulting in 402 single-label instances. Finally, we exclude nine non-informative comments consisting only of a generic *TODO* marker, such as *TODO*, *TODO:*, and *## TODO:*, because they provide no textual evidence for fine-grained classification. The final dataset therefore contains 393 SATD comments covering all 23 ML-specific SATD types. The distribution of the processed dataset is presented in Figure 2, showing a pronounced class imbalance, with 88 instances in the largest category and only one or two instances in several minority categories.

3.2 Models

To evaluate the effectiveness of *general-purpose* and *code-oriented* LLMs for fine-grained ML-SATD classification, we select models from three widely used open-weight families, namely Qwen, Gemma, and Llama, and additionally include the recently released open-weight GPT-oss model. Concretely, our evaluated models include Qwen3-8B, Qwen3-14B, Qwen2.5-Coder-7B-Instruct, gemma-2-9b-it, gemma-3-12b-it, codegemma-1.1-7b-it, Llama-3.1-8B-Instruct, CodeLlama-7b-Instruct-hf, and gpt-oss-20b. This selection enables a controlled comparison between general LLMs and code-specialized

LLMs under similar parameter scales, and supports reproducibility by relying on open-weight checkpoints.

Qwen3 series. Qwen3 [31] is the latest generation of the Qwen model family, providing both dense and Mixture-of-Experts (MoE) variants across a wide range of scales and emphasizing strong reasoning and multilingual coverage. A key design is the unified support of *thinking* and *non-thinking* modes, allowing the model to trade off reasoning depth and efficiency via an adjustable budget mechanism. In our experiments, we use the instruction-tuned dense variants Qwen3-8B and Qwen3-14B.

Qwen2.5-Coder. Qwen2.5-Coder [10] is a code-specific model family built upon the Qwen2.5 architecture and further pretrained on large-scale code-centric corpora (over trillions of tokens), aiming to improve code generation and code reasoning while preserving general and mathematical abilities. We include Qwen2.5-Coder-7B-Instruct to represent a strong open code LLM at a moderate scale.

Gemma 2 series. Gemma 2 [29] is a family of open-weight, decoder-only Transformer models (2B–27B) that improves efficiency and long-context modeling by interleaving local and global attention and adopting grouped-query attention. Smaller variants (including 9B) are trained with distillation to achieve competitive performance at practical sizes. We evaluate gemma-2-9b-it as the general instruction-tuned representative from this family.

Gemma 3 series. Gemma 3 [28] extends the Gemma family with longer context support (at least 128K tokens) and improved multilingual capabilities, and it is trained with distillation and a revised post-training recipe to enhance instruction following and reasoning. We use gemma-3-12b-it as a newer generation general instruction-tuned model with a comparable parameter scale to other backbones in our study.

CodeGemma. CodeGemma [27] is a collection of open code models built on top of Gemma, further trained on large code-heavy corpora to strengthen code understanding and generation. We include the instruction-tuned codegemma-1.1-7b-it as the code-oriented counterpart within the Gemma ecosystem.

Llama 3.1 series. Llama 3.1 [8] is an open foundation model family optimized for multilingual understanding, reasoning, and coding. It is trained on large-scale high-quality tokens and released with instruction-tuned variants for interactive use. We evaluate Llama-3.1-8B-Instruct as a widely adopted open general-purpose LLM baseline.

CodeLlama. CodeLlama [21] is a family of code-specialized models derived from Llama, designed for code generation and infilling, and provided with instruction-following variants for programming tasks. We include CodeLlama-7b-Instruct-hf to study whether code specialization in the Llama ecosystem benefits ML-SATD classification.

gpt-oss-20b. gpt-oss-20b [1] is an open-weight decoder-only Mixture-of-Experts model designed for efficient reasoning and tool use, and released with an instruction-tuned chat format that supports adjustable reasoning effort. We include gpt-oss-20b to compare a recent open-weight MoE backbone against mainstream dense open models under the same evaluation protocol.

Overall, by evaluating both general and code-oriented backbones across multiple open model families, our study provides practical guidance on whether (and when) code specialization influences ML-SATD classification performance. For all tested models, we use the default settings and set all temperatures to zero for reproducibility.

3.3 Evaluation Metrics

Following prior work on SATD classification [9], we evaluate our models using precision (P), recall (R), and F1-score. In the multi-class setting, each SATD category c_k is treated as a separate class. For each class c_k, we define the following quantities based on the multiclass confusion matrix:

- TP_k (true positives): the number of instances whose true label and predicted label are both c_k;
- FP_k (false positives): the number of instances whose predicted label is c_k, but whose true label is not c_k;
- FN_k (false negatives): the number of instances whose true label is c_k, but whose predicted label is a different class.

The class-wise precision, recall, and F1-score are then defined as:

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad R_k = \frac{TP_k}{TP_k + FN_k}, \quad F1_k = \frac{2P_kR_k}{P_k + R_k}. \tag{1}$$

Since our task is multi-class and the class distribution is highly imbalanced, we report *weighted averages* of precision, recall, and F1-score. Let n_k be the number of test instances (support) of class c_k, and $N = \sum_{k=1}^{K} n_k$ be the total number of test instances. The weighted-averaged precision, recall, and F1-score are defined as:

$$\text{Weighted-Precision} = \sum_{k=1}^{K} \frac{n_k}{N} P_k, \quad \text{Weighted-Recall} = \sum_{k=1}^{K} \frac{n_k}{N} R_k, \quad \text{Weighted-}F1 = \sum_{k=1}^{K} \frac{n_k}{N} F1_k. \tag{2}$$

These weighted averages correspond to the standard implementation in commonly used libraries (e.g., scikit-learn) and reflect performance under the empirical class distribution of the ML-SATD dataset.

3.4 Methodology for RQ1

We test three different prompting strategies to identify the most effective way to instruct the LLM for fine-grained ML SATD classification: vanilla prompting, binary prompting, and hierarchical prompting. For vanilla prompting, we enumerates all target categories and instructs the model to output only one of the corresponding label abbreviation.

The prompts used in this method are consist of two parts: 1) catergories defination. which consist of the detailed defination of the labels. 2) Task instruction. Which are the detailed instruction of different prompting. Vanilla prompting enumerates all target categories and instructs the model to output only one of the corresponding label abbreviation. Binary prompting is a two-stage method. In the first stage, the model is asked for each type of defined ML SATD whether the given code comment belongs to that type. After this step, the model may either select multiple types or fail to select any. In the second stage, these issues are resolved: if the model returns multiple types, it is asked to choose the most likely one from the candidates; if it fails to select any, it is asked again to select from the full set of categories. Hierarchical prompting is also a two-stage method. Since the dataset contains both high-level categories and subcategories, the classification task is organized into a tree structure. The model first determines the high-level category of the ML SATD and then proceeds step by step to fine-grained subcategories until the final label is reached.

3.5 Methodology for RQ2

Few-shot examples can improve models' performance in other tasks, however, Due to the limited size of the dataset in this task, some categories contain only a single instance, which makes it difficult to devide the dataset and select representative examples. In this RQ, we aim to test whether the model generated examples can improve the results. We use an LLM to generate additional examples. Specifically, we provide the LLM with detailed category descriptions and real word examples to prompt these LLMs to generate high-quailty examples. Then we manually verified the quaility of these generated examples, we focus on two aspect, 1) to avoid data leakage, make the

Table 1. Model-generated examples of the 23 Identified ML SATD Types.

Dim. Name	Type	Example
AWR	Machine Learning Knowledge	TODO: Confirm whether layer-norm before attention actually boosts convergence for small-batch training on this dataset
	Model Interpretability	TODO: Generate SHAP summary plot so users can see feature impact
RDB	Model Code Comprehension	TODO: break up monolithic function train_eval_predict() into clearer, single-purpose helpers
DCE	Duplicate Model Code	TODO: remove copy-pasted ConvBlock definition—reuse common_layers.ConvBlock instead
	Duplicate Feature Extraction Code	XXX: text_features_v1 and text_features_v2 are copy-pasted—create one configurable pipeline
CFO	Weight Configuration	FIXME: pretrained checkpoint loads fp16 weights—add auto-cast to ensure dtype consistency
	Layer Configuration	TODO: swap BatchNorm for LayerNorm when depth > 24
	Hyper-parameter Configuration	TODO: make learning_rate schedule (cosine vs. linear) selectable via CLI flag
CDD	Machine Learning Dependency	TODO: bump scikit-learn to ≥ 1.6 once conda forge wheels for arm64 become available
	Glue Code	FIXME: adapter.py only exists to map old Predictor API to new one—rewrite consumers instead
	Custom Data Type	TODO: replace custom SparseTensorDict with standard torch.sparse_coo_tensor before exporting
	Multiple Languages	TODO: port custom attention kernel from CUDA C++ to Metal for macOS deployment
	Unnecessary Model Code	TODO: delete legacy compute_grad_stats(); superseded by torch.autograd.profiler
DTD	Data Processing Configuration	TODO: add configurable pipeline step to down-sample majority class before batching
	Plain Old Data Type	TODO: replace raw list of logits with a NamedTuple so callers know the order
	Data Storage Configuration	TODO: migrate checkpoint format from .pth to safetensors for safer serialization
MDL	Abstraction	TODO: extract common scoring logic into BaseMetric so each model wrapper isn't re-implementing it
	Boundary Erosion	TODO: migrate training-time logging out of Trainer class; should live in metrics.logger
	Model Code Modifiability	TODO: move hyper-parameter defaults into config file so changing them doesn't require code edits
	Model Code Reusability	TODO: abstract make_encoder() so VisionTransformer and ResNet builders share the same backbone factory
PRF	Prediction Quality	TODO: rerun validation after enabling mixed-precision; confirm accuracy isn't degraded
	Machine Learning Reliability	TODO: add sanity-check to reject examples with > 20% missing features before prediction
SCL	Prototype	TODO: replace the single-threaded demo server with a production-ready Gunicorn setup

generated examples is not too similar to the test case, we using the 2) to ensure the diversitries,

```
You are a strict ML-SATD classifier.

Decide the single best label by following this internal decision process, then output exactly ONE abbreviation
      on a single line with nothing else.

Internal decision process (for your internal use only; do not include in output):

1) Identify the primary signal in the comment and select candidates accordingly:

- Data-related: processing/pipeline steps => {{Data Conf}}; raw/plain data types => {{PODT}}; representation/
     storage/schema => {{DS Conf}}.
- Dependency & integration: external/library blocking => {{MLDepend}}; glue/integration shims => {{GC}};
     custom data types => {{CDT}}; mixed languages => {{ML}}; unnecessary/dead/deprecated model code => {{UMC
     }}.
- Configuration & parameters: weights => {{Weight Conf}}; layers/topology => {{Layer Conf}}; hyper-parameters/
     defaults => {{HP Conf}}.
- Structure & evolvability: missing abstraction => {{Abs}}; boundary erosion => {{BE}}; ease of modifying
     model code => {{MCM}}; generality/reuse => {{MCR}}.
- Duplication: model code duplication => {{DMC}}; feature-extraction duplication => {{DFEC}}.
- Understanding & interpretability: lack of ML knowledge => {{MLKnow}}; model black-box / explainability
     concerns => {{MLInter}}.
- Performance & robustness: prediction quality/accuracy => {{PQ}}; reliability/robustness/data-quality guards
     => {{MLRelia}}.
- Prototype: prototype used beyond its scope => {{Proto}}.

2) If multiple candidates match, choose the one that represents the main maintenance obstacle implied by the
     comment.

3) If still tied, prefer the most specific label whose remediation is the most localized (e.g., pick {{Layer
     Conf}} over {{HP Conf}} if the text explicitly mentions layers).

4) Output policy: return exactly ONE abbreviation on a single line, with no other text.

Example output: Data Conf
Allowed abbreviations: Abs, BE, CDT, Data Conf, DFEC, DMC, DS Conf, GC, HP Conf, Layer Conf, MCC, MCM, MCR, ML,
      MLDepend, MLInter, MLKnow, MLRelia, PODT, PQ, Proto, UMC, Weight Conf
```

Fig. 3. Details of Chain-of-Thought instructions used in the experiments.

we manually drop some generated examples are too similarly to other examples in the generated examples set.

up to five real examples per category—using fewer examples if fewer than five instances exist—and instruct it to generate similar examples.

3.5.1 Model Choice. We adopt GPT-5, a state-of-the-art model, in deep research mode to generate these examples. We generate ten synthetic examples for each category and incorporate them into the instruction. We provide an example for each category in Table 1. It should be noted that the generated examples are fixed for each instance during inference. To examine the impact of the number of examples, we evaluate settings with 1, 2, 3, 5, and 10 examples. We incorporate these examples into the two best performing prompts found in RQ1.

3.5.2 Examples Selection. After get the model generated examples, we manually verified and selected the examples, we find for most of the catergories, GPT-5 can generate suitable examples, but for the generated examples some are very similar, for this situation, we using GPT-5 to regenerate it again, all the examples are different in the second generation.

After get the generate examples, we matain 10 examples of each categories and

3.6 Methodology for RQ3

To test the models' ability under thinking mode, Qwen series and GPT-OSS-20b are designed for thinking capability, so to test these models, we set these models according to the guidelines on the website to complete this task. For the Qwen series, we set *enable_thinking* equal to true. For GPT-OSS-20b, we evaluate our task under high mode which the high mode delivers deep and comprehensive analysis.

We first compared the Qwen series under different settings which is thinking and no-thinking mode to figure out the effectiveness of thinking of these models.

We want to know whether the manually written Chain-of-Thought can improve the performance, so we first define the patterns in ML piplines to map the detailed patterns to the detailed sub catergoies. This information can give a detailed information about how these

The thinking models We also compared the reasoning mode with the zero-shot Chain-of-Thought [11], which requires the model to think step by step.

The Qwen series can be

Previous models, which didn't include the thinking mode, were tested under chain-of-thought settings and compared with those models under the thinking mode. The thinking models are We also test the reasoning mode under chain-of-thought prompting in two steps, the first one following traditional zero-shot prompting requires model to think step by step and output reasoning and the answers. Beside the traditional zero-shot prompting, we refine these prompts by adding more detailed chain of reasoning, we manually defined the features which will lead to reason about each types. We test these refined Chain-of-thought prompting and analysis the thinking models between the chat models.

3.7 Methodology for RQ4

RQ4 aimed to examine the overall effectiveness of LLMs on the fine-grained ML SATD classification task after combining the effective prompting strategies identified in the previous RQs. To this end, we conducted an additional round of experiments based on the results of RQ1–RQ3, to explore whether different prompting strategies combinations could further improve model performance.

We first reviewed the results obtained in the previous RQs and selected the three most promising models for further analysis. The selection was based on their overall performance under different prompting settings, including the prompt formulations studied in RQ1, the example-based settings explored in RQ2, and the reasoning strategies examined in RQ3. Rather than including all evaluated models, we focused on a smaller set of representative and competitive models in order to more clearly assess the effect of combining prompting strategies.

Based on the findings of the previous RQs, we then constructed the combined settings used in this RQ. Specifically, we adopted the best-performing prompt setting identified in RQ1 as the basis of the final experiments. On top of this setting, we incorporated the automatically generated examples investigated in RQ2 and the manually written chain-of-thought explored in RQ3. This design allowed us to evaluate whether the improvements observed in earlier RQs could be accumulated when these strategies were applied together.

In addition to the LLM-based settings, we included two simple baselines to better contextualize the final results. The first baseline was a random-guess baseline, which assigned one of the 23 ML-SATD labels uniformly at random to each instance. Since this baseline was stochastic, we repeated it 10 times with different random seeds and reported the average results together with the standard deviation. The second baseline was a majority-class baseline, which always predicted the most frequent label in the dataset. Both baselines were evaluated on the same dataset using the same weighted precision, weighted recall, and weighted F1-score as the LLM-based approaches.

The analysis in RQ4 focused on three aspects. First, we compared the LLM-based approaches with the two baselines in order to assess whether prompted LLMs could outperform simple non-LLM prediction strategies. Second, we examined whether combining prompting strategies led to further improvements over the individual settings studied in RQ1–RQ3. Third, we identified the best-performing model and prompting strategies among all evaluated settings, thereby providing a clearer picture of the performance that current LLMs can achieve on fine-grained ML SATD classification.

4 RESULTS

4.1 RQ1: How to prompt LLMs for fine-grained ML SATD classification?

Results of different prompting strategies. We compare the three prompting methods introduced above, and report the detailed zero-shot results in Table 2.

Under the vanilla prompt, gpt-oss-20b achieves the best overall performance with an F1 score of 34.0% (precision 0.434, recall 0.344). Qwen3-14B ranks second (30.2% F1), while gemma-3-12b places third (28.9% F1). Overall, the vanilla setting yields relatively balanced precision-recall trade-offs for the top performance models.

With the binary prompt, performance generally improves over vanilla for most models. gpt-oss-20b remains the top model and further increases its performance to 35.1% F1 (precision 0.397, recall 0.354). gemma-3-12b becomes the second-best model with 32.6% F1, followed by Qwen2.5-Coder-7B-Instruct with 31.1% F1. This may be because the binary prompting gives the LLMs a second chance to select the more precise results from the smaller candidate pools, which makes some model performance better.

For the hierarchical prompt, we observe a clear decline in overall results compared to the other two prompting strategies. Although some models obtain higher precision (e.g., gemma-2-9b reaches 0.460 precision), recall drops sharply across most models, leading to lower F1 scores. gpt-oss-20b still performs best with 32.6% F1, but the gap to the remaining models narrows, with Qwen3-14B at 25.1% F1 and gemma-2-9b at 23.9% F1. These results indicate that, in a zero-shot setting, hierarchical decomposition can introduce additional error propagation across stages, making the overall prediction less accurate than the simpler vanilla or binary prompts.

Summary of different prompting strategies. Among the nine models evaluated, the vanilla prompting and binary classification prompting achieve comparable performance, while the hierarchical prompting performs the worst. Comparing the vanilla and hierarchical prompting approaches, we find that eight out of nine models perform better in terms of F1 score under the vanilla setting. A similar trend is observed for the binary prompting, where all models outperform those using the hierarchical approach. We attribute this to the difficulty LLMs face in understanding higher-level hierarchical labels, which may lead to error accumulation when predicting the final results, ultimately degrading performance. Comparing the vanilla prompting with the binary prompting, we find that these two approaches yield comparable performance, with the binary approach performing slightly better. Specifically, six out of nine models perform better under the binary setting than under the vanilla one. We believe this is because both approaches do not introduce additional information that could increase the model's cognitive burden. Moreover, the binary approach allows the model to group uncertain types and provides a second opportunity to make a more accurate prediction.

Error analysis. We provide detailed analysis of the hierarchical prompting

Table 2. Zero-shot results on three prompting methods

Model	Vanilla			Binary			Hierarchical		
	precision	recall	f1	precision	recall	f1	precision	recall	f1
Qwen3-8B	0.210	0.196	0.168	0.315	0.237	0.249	0.263	0.198	0.170
Qwen3-14B	0.379	0.305	0.302	0.387	0.280	0.284	0.452	0.244	0.251
Qwen2.5-Coder-7B-Instruct	0.315	0.262	0.242	0.406	0.323	0.311	0.398	0.221	0.210
gemma-2-9b	0.370	0.290	0.268	**0.407**	0.293	0.279	**0.460**	0.270	0.239
gemma-3-12b	0.412	0.285	0.289	0.404	0.328	0.326	0.309	0.272	0.228
codegemma-1.1-7b-it	0.308	0.198	0.167	0.291	0.183	0.171	0.296	0.094	0.083
Llama-3.1-8B-Instruct	0.231	0.221	0.156	0.301	0.153	0.146	0.219	0.089	0.095
CodeLlama-7b-Instruct-hf	0.214	0.148	0.108	0.109	0.048	0.052	0.077	0.028	0.017
gpt-oss-20b	**0.434**	**0.344**	**0.340**	0.397	**0.354**	**0.351**	0.434	**0.310**	**0.326**

Findings: (1) For fine-grained ML SATD classification, the vanilla prompting and the binary prompting achieve similar results, and the hierarchical prompting is the worst.
(2) GPT-oss-20b under the binary prompting setting is the best model for fine-grained ML SATD classification under zero-shot settings.

4.2 RQ2: Can incorporating LLM-generated examples improve the performance of LLMs on fine-grained ML SATD classification?

We add the examples into two prompting strategies which are were revealed in RQ1, that is Vanilla prompting and Binary prompting.

Overall effectiveness of adding examples. Incorporating LLM-generated examples can improve performance. The detailed results are shown in Table 3 and Table 4. Compared with the zero-shot results on this task, under the vanilla prompting setting, we find that all models achieve better results than in the zero-shot setting. The same trend is also observed under the binary prompting setting, where seven out of nine models perform better than in the zero-shot setting.

Under the vanilla prompting, adding LLM-generated examples is beneficial for most models. Specifically, 7 out of 9 models achieve a higher F1 score when providing one or more examples than in the zero-shot setting, indicating that adding examples is generally effective. Among them, Qwen3-8B shows the largest improvement, with its F1 score increasing from 16.8% in the zero-shot setting to 25.6% with 5 examples, yielding an absolute gain of 8.8%. The second- and third-largest improvements are observed for Llama-3.1-8B-Instruct and codegemma-1.1-7b-it, whose F1 scores improve by 6.7% and 5.3%, respectively. In contrast, Qwen2.5-Coder-7B-Instruct and gemma-3-12b do not benefit from adding examples, as none of their few-shot results surpass their corresponding zero-shot performance. In terms of the best overall performance achieved under this setting, the top three models are gpt-oss-20b, which achieves the highest F1 score of 36.9% with 2 examples, followed by Qwen3-14B with 33.8% at 3-shot, and gemma-2-9b with 29.8% at 2-shot.

Under the binary prompting setting, adding in-context examples is also beneficial for most models. Specifically, 7 out of 9 models achieve a higher F1 score in at least one few-shot configuration than in the 0-shot setting, indicating that example augmentation remains generally effective under this prompting strategy. Among all models, Llama-3.1-8B-Instruct shows the largest improvement, with its F1 score increasing from 14.6% in the 0-shot setting to 22.6% with 2 examples, yielding an

Table 3. Few-shot result under the vanilla prompting setting

		Qwen3-8B	Qwen3-14B	Qwen2.5-Coder-7B-Instruct	gemma-2-9b	gemma-3-12b	codegemma-1.1-7b-it	Llama-3.1-8B-Instruct	CodeLlama-7b-Instruct-hf	gpt-oss-20b
0	precision	0.210	0.379	0.315	0.370	0.412	**0.308**	0.231	**0.214**	**0.434**
	recall	0.196	0.305	**0.262**	0.290	0.285	0.198	0.221	0.148	0.344
	f1	0.168	0.302	**0.242**	0.268	**0.289**	0.167	0.156	0.108	0.340
1	precision	0.246	0.388	0.294	0.337	0.407	0.256	**0.298**	0.186	0.397
	recall	0.232	0.275	0.244	**0.310**	**0.303**	0.229	0.214	0.142	0.356
	f1	0.208	0.287	0.230	0.275	0.263	0.195	0.189	**0.140**	0.346
2	precision	0.256	0.419	0.297	**0.378**	0.401	0.284	0.237	0.189	0.431
	recall	0.249	0.313	0.229	0.303	0.295	**0.265**	0.206	0.183	**0.379**
	f1	0.238	0.328	0.213	**0.298**	0.260	**0.220**	0.191	0.123	**0.369**
3	precision	0.255	**0.422**	**0.345**	0.359	**0.435**	0.243	0.270	0.126	0.415
	recall	0.260	**0.321**	0.234	0.298	0.275	0.237	**0.226**	0.186	0.344
	f1	0.234	**0.338**	0.231	0.278	0.234	0.195	**0.223**	0.124	0.345
5	precision	**0.302**	0.368	0.293	0.355	0.406	0.251	0.250	0.144	0.402
	recall	**0.280**	0.272	0.209	0.277	0.282	0.201	0.196	0.168	0.344
	f1	**0.256**	0.287	0.210	0.268	0.247	0.184	0.189	0.107	0.348
10	precision	0.254	0.370	0.298	0.298	0.205	0.252	0.270	0.050	0.427
	recall	0.224	0.277	0.204	0.234	0.226	0.191	0.201	**0.224**	0.349
	f1	0.222	0.287	0.206	0.220	0.170	0.158	0.185	0.082	0.352

Table 4. Few-shot result under the binary prompting setting

		Qwen3-8B	Qwen3-14B	Qwen2.5-Coder-7B-Instruct	gemma-2-9b	gemma-3-12b	codegemma-1.1-7b-it	Llama-3.1-8B-Instruct	CodeLlama-7b-Instruct-hf	gpt-oss-20b
0	precision	0.315	0.387	**0.406**	0.407	**0.404**	0.291	0.301	0.109	0.397
	recall	0.237	0.280	**0.323**	0.293	0.328	0.183	0.153	0.048	0.354
	f1	0.249	0.284	**0.311**	0.279	**0.326**	0.171	0.146	0.052	0.351
1	precision	0.316	0.399	0.333	0.404	0.404	**0.379**	0.276	0.138	**0.412**
	recall	0.265	0.282	0.298	**0.300**	**0.338**	**0.234**	**0.211**	0.061	**0.356**
	f1	0.258	0.302	0.282	**0.312**	0.324	**0.218**	0.203	0.067	**0.357**
2	precision	**0.352**	**0.456**	0.354	0.416	0.383	0.281	0.369	0.147	0.397
	recall	0.267	0.316	0.321	0.298	0.326	0.219	0.209	0.079	0.351
	f1	0.281	0.335	0.298	0.309	0.305	0.190	**0.226**	0.075	0.348
3	precision	0.333	0.449	0.347	0.416	0.363	0.232	0.374	0.270	0.395
	recall	**0.285**	0.316	0.293	0.290	0.300	0.209	0.201	0.089	0.354
	f1	**0.284**	0.337	0.272	0.305	0.288	0.149	0.207	0.092	0.351
5	precision	0.346	0.452	0.377	**0.419**	0.402	0.287	0.348	**0.309**	0.405
	recall	0.262	**0.318**	0.288	0.277	0.308	0.224	0.181	0.097	0.344
	f1	0.267	**0.342**	0.286	0.289	0.309	0.175	0.184	0.084	0.341
10	precision	0.340	0.429	0.264	0.312	0.397	0.261	**0.379**	0.123	0.381
	recall	0.234	0.285	0.183	0.186	0.270	0.173	0.191	**0.122**	0.338
	f1	0.250	0.301	0.189	0.191	0.282	0.156	0.205	**0.099**	0.334

absolute gain of 8.0 percentage points. The second-largest improvement is observed for Qwen3-14B, whose F1 score improves from 28.4% to 34.2% at 5-shot, corresponding to an increase of 5.8 percentage points. codegemma-1.1-7b-it and CodeLlama-7b-Instruct-hf also show notable gains, with both improving by 4.7 percentage points over their 0-shot results. In contrast, Qwen2.5-Coder-7B-Instruct and gemma-3-12b do not benefit from adding examples, as none of their few-shot results exceed their corresponding 0-shot performance. In terms of the best overall performance achieved under this setting, the top three models are gpt-oss-20b, which attains the highest F1

Table 5. Results on coarse-grained categories under the hierarchical setting

Model	precision	recall	f1
Qwen3-8B	0.405	0.254	0.214
Qwen3-14B	0.517	0.318	0.350
Qwen2.5-Coder-7B-Instruct	0.484	0.331	0.319
gemma-2-9b	0.519	0.366	0.364
gemma-3-12b	0.511	0.341	0.333
codegemma-1.1-7b-it	0.503	0.176	0.182
Llama-3.1-8B-Instruct	**0.548**	0.117	0.127
CodeLlama-7b-Instruct-hf	0.096	0.033	0.026
gpt-oss-20b	0.530	**0.392**	**0.420**

score of 35.7% with 1 example, followed by Qwen3-14B with 34.2% at 5-shot, and gemma-3-12b with 32.6% in the 0-shot setting.

Impact of number of examples. We further analysis the impact of the numbers of examples, from Table 3 and Table 4, we find that adding more examples into the prompts don't lead to improved performance. Under the vanilla setting, we find that most models achieve their best few-shot performance when provided with fewer than three examples. Specifically, 6 out of 9 models reach their highest F1 scores at either 1-shot or 2-shot, including Qwen2.5-Coder-7B-Instruct, gemma-2-9b, gemma-3-12b, codegemma-1.1-7b-it, CodeLlama-7b-Instruct-hf, and gpt-oss-20b. Qwen3-14B and Llama-3.1-8B-Instruct achieve their best results at 3-shot, while Qwen3-8B is the only model whose best performance appears at 5-shot. Importantly, increasing the number of examples to 10 does not further improve any model under vanilla prompting, and in most cases leads to a decline compared with the best few-shot setting. For example, gemma-2-9b drops from 29.8% at 2-shot to 22.0% at 10-shot, codegemma-1.1-7b-it decreases from 22.0% to 15.8%, and CodeLlama-7b-Instruct-hf falls from 14.0% at 1-shot to 8.2%.

A similar trend can also be observed under the binary prompting setting. Again, 6 out of 9 models achieve their best few-shot performance with only 1 or 2 examples, including Qwen2.5-Coder-7B-Instruct, gemma-2-9b, gemma-3-12b, codegemma-1.1-7b-it, Llama-3.1-8B-Instruct, and gpt-oss-20b. Qwen3-8B reaches its best result at 3-shot, Qwen3-14B at 5-shot, and CodeLlama-7b-Instruct-hf is the only model whose best few-shot result appears at 10-shot. However, even in this case, the overall performance remains relatively low. More importantly, larger numbers of examples are usually not helpful: for instance, gpt-oss-20b declines from 35.7% at 1-shot to 33.4% at 10-shot, Qwen3-14B drops from 34.2% at 5-shot to 30.1%, and Qwen2.5-Coder-7B-Instruct decreases substantially from 29.8% at 2-shot to 18.9%.

Overall, these results indicate that the performance gain from few-shot prompting is not increased with the number of examples. Across the two prompting settings, 12 out of 18 model–prompt combinations achieve their best few-shot performance with only 1 or 2 examples, and 15 out of 18 achieve their best result with no more than 3 examples. This suggests that a small number of examples is usually sufficient to provide useful task guidance, while adding too many examples may instead introduce noise and increase the cognitive burden of the model.

Findings: (1)Incorporating the LLM-generated examples can improve the performance. (2) Not incorporating more examples yields better results, and we find that adding fewer than three examples can achieve better results.

4.3 RQ3: Can reasoning improve the performance of LLMs on fine-grained ML SATD classification?

Comparison between Thinking and no-Thinking mode. Table 6 compares the effect of enabling thinking mode under both Vanilla and Binary prompting. Overall, enabling thinking consistently improves performance across all models, indicating that these reasoning model allowing them to performance deep analysis of the task and find some useful patterns to get better results beneficial for fine-grained ML-SATD classification. Under Vanilla prompting, the gains are particularly substantial. For Qwen3-8B, enabling thinking improves precision from 0.210 to 0.403, recall from 0.196 to 0.326, and F1 from 0.168 to 0.303, yielding an absolute F1 gain of 13.5 percentage points. Qwen3-14B also benefits from thinking under Vanilla prompting, with F1 increasing from 0.302 to 0.341. These results suggest that reasoning helps the model better interpret the label semantics and make more accurate decisions when the prompt.

Under Binary prompting, thinking still brings improvements, but the gains are noticeably smaller than those under Vanilla prompting. For Qwen3-8B, thinking raises F1 from 0.249 to 0.282, while for Qwen3-14B, the improvement is more limited, from 0.284 to 0.300. A closer look shows that the main benefit of thinking in the Binary setting lies in precision rather than recall. For example, Qwen3-14B improves precision from 0.387 to 0.467, but recall increases only slightly from 0.280 to 0.308. This indicates that, under Binary prompting, thinking makes the model more cautious and selective, which improves correctness of positive predictions but does not substantially expand coverage. In summary, thinking is consistently helpful, but its effect is stronger under Vanilla prompting, especially for the smaller model, suggesting that reasoning is most valuable when the prompt leaves more room for the model to structure its own decision process.

Table 6. Thinking vs No-Thinking

Models	Metric	Vanilla		Binary	
		no-thinking	thinking	no-thinking	thinking
Qwen3-8B	precision	0.210	**0.403**	0.315	0.398
	recall	0.196	**0.326**	0.237	0.267
	f1	0.168	**0.303**	0.249	0.282
Qwen3-14B	precision	0.379	0.420	0.387	**0.467**
	recall	0.305	**0.366**	0.280	0.308
	f1	0.302	**0.341**	0.284	0.300

Effectiveness of incorporating CoT. Table 7 shows the effectiveness of adding manually summarized Chain-of-Thought. Compared with the corresponding zero-shot results in Table 2, the performance of most models improves after incorporating the reasoning prompt. Under the vanilla prompting setting, 6 out of 9 models achieve higher F1 scores than in the original zero-shot setting. The largest improvement is observed for Qwen3-8B, whose F1 score increases from 16.8% to 29.5%, yielding an absolute gain of 12.7 percentage points. Qwen3-14B and codegemma-1.1-7b-it also show

Table 7. Results of different models under Vanilla and Binary prompting settings

Model	Vanilla_CoT			Binary_CoT		
	precision	recall	f1	precision	recall	f1
Qwen3-8B	0.356	0.333	0.295	0.389	0.349	0.333
Qwen3-14B	**0.472**	**0.384**	**0.363**	0.400	0.336	0.313
Qwen2.5-Coder-7B-Instruct	0.351	0.310	0.289	0.407	0.316	0.300
gemma-2-9b	0.355	0.288	0.275	0.382	0.295	0.297
gemma-3-12b	0.390	0.285	0.270	0.381	0.290	0.292
codegemma-1.1-7b-it	0.394	0.239	0.218	**0.450**	0.244	0.231
Llama-3.1-8B-Instruct	0.213	0.178	0.141	0.321	0.155	0.155
CodeLlama-7b-Instruct-hf	0.161	0.102	0.071	0.135	0.066	0.065
gpt-oss-20b	0.427	0.359	0.345	0.447	**0.372**	**0.362**

clear improvements, with F1 gains of 6.1 and 5.1 percentage points, respectively. Under the binary prompting setting, the manually written CoT is beneficial for 7 out of 9 models. Again, Qwen3-8B shows the largest improvement, with its F1 score increasing from 24.9% to 33.3%, followed by codegemma-1.1-7b-it, whose F1 score rises from 17.1% to 23.1%. These results indicate that manually written CoT is generally effective for fine-grained ML-SATD classification, especially when the task requires distinguishing highly similar labels.

Compare the Qwen series under the thinking mode and those traditional LLMs argument by the with the manually written CoT easoning prompt, we find that Qwen series model's performance increased compared to the one without thinking. Qwen3-series outperform the

Case Study. We further analysis the impact of these

> **Findings:** (1) Reasoning mode can improve the performance, and incorporating the manually written Chain-of-thought can further improve the performance.
> (2) Reasoning models are the best choice for fine-grained ML SATD classification task; they are better than traditional models and even incorporate the manually written Chain-of-thought, it still surpasses these models.

Effectiveness of incorporating CoT. Comparing the Qwen series under thinking mode with those using manually written CoT, we find that manually summarized reasoning instructions remain helpful even for reasoning-capable models. For Qwen3-14B, incorporating CoT further improves performance in both prompting settings, with F1 increasing from 34.1% to 36.3% under vanilla prompting and from 30.0% to 31.3% under binary prompting. For Qwen3-8B, manually written CoT slightly underperforms native thinking under vanilla prompting (29.5% vs. 30.3%), but leads to a much stronger result under binary prompting, where F1 increases from 28.2% to 33.3%. This suggests that manually summarized CoT and native thinking are not redundant; instead, they can provide complementary benefits, especially when the prompt structure constrains the reasoning process. Overall, the Qwen3-series remains among the strongest models in this setting, which further confirms that reasoning-capable models are particularly suitable for fine-grained ML-SATD classification.

Findings: (1) Reasoning mode can improve performance, and incorporating manually written Chain-of-Thought can further improve the results for most models.
(2) Reasoning-capable models are the best choice for fine-grained ML SATD classification. Even without manually written CoT, they already outperform most traditional models, and with additional reasoning guidance, they achieve even stronger performance.

4.4 RQ4: How effective are LLMs on fine-grained classification of ML SATD?

Baseline results. To better understand the effectiveness of LLMs on this task, we first compare them with two simple baselines, namely random guess and majority-class prediction. For the random-guess baseline, the predictor randomly selects one of the classes in the 23 label types. We repeated the experiment 10 times with different random seeds and report the average results together with the standard deviation. The random-guess baseline achieves a weighted precision of 0.109 ± 0.034, a weighted recall of 0.044 ± 0.013, and a weighted F1-score of 0.054 ± 0.016.

For the majority-class baseline, which predicts the most frequent label in the dataset, namely *Data Processing Configuration*. Under this setting, the majority-class baseline achieves a weighted precision of 0.050, a weighted recall of 0.224, and a weighted F1-score of 0.082. Compared with random guessing, the majority-class baseline yields a noticeably higher recall and F1-score, which reflects the imbalanced label distribution of the dataset.

Not all models under the different settings explored in RQ1-RQ3 outperform the random-guess baseline or the majority-class baseline. For example, CodeLlama-7b-Instruct-hf under the hierarchical prompting setting performs the worst and even falls below random guessing. This shows the importance of model selection and prompting strategies.

Combination results. Table 8 presents the results obtained by combining the effective prompting strategies identified in the previous RQs. Overall, the results show that the combination of prompting strategies can further improve model performance on fine-grained ML SATD classification. In particular, using examples together with manually written chain-of-thought generally leads to better results than the corresponding zero-shot setting, although the improvement varies across models and prompting strategies.

Considering weighted F1-score as the main evaluation metric, the best overall result is achieved by **Qwen3-8B** under **vanilla prompting** with **three examples**, reaching an F1-score of **0.381**. This is also the highest F1-score among all settings explored in our study. The result slightly surpasses the best performance of **gpt-oss-20B**, which achieves an F1-score of 0.375 under vanilla prompting with three examples, and also exceeds the strongest binary-prompting results, where both Qwen3-8B and Qwen3-14B reach 0.377 with three examples.

The effect of adding examples is generally positive. Under vanilla prompting, the F1-score of Qwen3-8B increases steadily from 0.295 with zero examples to 0.381 with three examples, showing the largest improvement among all configurations. A similar trend can also be observed for gpt-oss-20B, whose F1-score increases from 0.355 to 0.375. Under binary prompting, Qwen3-8B improves from 0.333 to 0.377, while Qwen3-14B improves from 0.313 to 0.377. For Qwen3-14B under vanilla prompting, however, the improvement is marginal, with the best result appearing at one example (0.365), followed by a slight decline when more examples are added. These results suggest that examples are generally beneficial, but the optimal number of examples still depends on the model and prompting style.

The comparison between vanilla and binary prompting also reveals model-specific differences in the final combined setting. For Qwen3-8B and gpt-oss-20B, vanilla prompting produces the best final results when combined with three examples and manually written chain-of-thought. In

contrast, Qwen3-14B performs better under binary prompting, where its best F1-score reaches 0.377. This finding indicates that the effectiveness of the final prompting combination is not determined by a single prompting style alone, but by the interaction between the model and the prompting design.

Taken together, these results suggest that the strategies explored in RQ1–RQ3 are complementary. The prompt setting identified in RQ1 provides an effective basis for the task, the examples examined in RQ2 help the models better capture fine-grained distinctions among ML-SATD labels, and the reasoning strategy studied in RQ3 further improves classification performance. By combining these strategies, we obtain the strongest results of the evaluated models on this task.

Table 8. Results of the combination of different prompting strategies

Setting	Metric	Vanilla prompting			Binary prompting		
		Qwen3-8B	Qwen3-14B	Gpt-oss-20B	Qwen3-8B	Qwen3-14B	Gpt-oss-20B
0	precision	0.356	**0.472**	0.420	0.389	0.400	0.447
	recall	0.333	**0.384**	0.377	0.349	0.336	**0.372**
	f1	0.295	0.363	0.355	0.333	0.313	0.362
1	precision	0.397	0.433	0.405	0.412	**0.447**	0.430
	recall	0.359	0.364	0.359	0.354	0.359	0.356
	f1	0.345	**0.365**	0.349	0.358	0.363	0.355
2	precision	0.430	0.438	**0.441**	0.422	0.426	0.408
	recall	0.366	0.372	0.359	0.364	0.366	0.364
	f1	0.351	0.363	0.358	0.360	0.361	0.354
3	precision	**0.445**	0.425	0.436	**0.424**	0.435	**0.450**
	recall	**0.389**	0.361	**0.382**	**0.382**	**0.382**	0.369
	f1	**0.381**	0.360	**0.375**	**0.377**	**0.377**	**0.371**

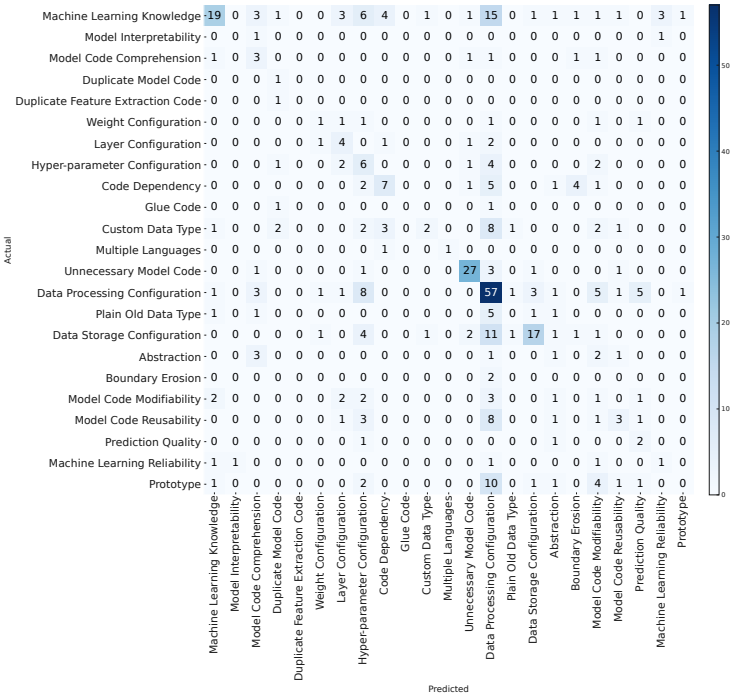


Fig. 4. Confusion Matrix for the best model.

We further analyze the confusion matrix of the best performing model, which is Qwen3-8B. As shown in the Figure 4, the model performs relatively well on several high-frequency categories, especially Data Processing Configuration, Unnecessary Model Code, Machine Learning Knowledge, and Data Storage Configuration, which form the most visible diagonal blocks in the matrix.

This pattern is consistent with the class distribution of the processed dataset, where these labels also have the largest supports. In particular, Data Processing Configuration shows the strongest diagonal count, indicating that the model can capture this category reasonably well under the best setting. At the same time, however, this label also acts as a major "attractor" class: many instances from Machine Learning Knowledge, Data Storage Configuration, Prototype, Custom Data Type, and Model Code Reusability are misclassified into it. This suggests that the model tends to over-predict broad data- and configuration-related debt when label boundaries are ambiguous. A similar confusion also appears among semantically adjacent configuration-related categories, such as Data Processing Configuration, Hyper-parameter Configuration, and Data Storage Configuration, indicating that although the model can recognize the general configuration nature of these comments, it still struggles to distinguish their finer-grained subtypes. In contrast, several minority or more abstract categories, such as Prototype, Plain Old Data Type, Model Interpretability, Glue Code, and Boundary Erosion, have very weak or even zero true positives, showing that current LLMs still have difficulty recognizing rare labels and categories with less explicit lexical cues. Overall, Figure 4 shows that the best model already captures the dominant structure of the label space, but its errors are concentrated on fine-grained distinctions among semantically overlapping ML-SATD types, which remains the main bottleneck for further performance improvement.

Findings: (1) Combining different prompting strategies can further improve the performance of LLMs on fine-grained ML SATD classification.
(2) While not all selected model settings in the earlier RQs outperform the majority-class baseline, all combined configurations in Table 8 achieve clearly better results than both the random-guess and majority-class baselines.
(3) The best-performing configuration is Qwen3-8B under vanilla prompting with three examples and manually written Chain-of-Thought, achieving the highest weighted F1-score of **0.381**.

5 DISCUSSION

In this section, we further summarize the practice guilderlines for using LLMs for fine-grained SATD classification and lessons learned from the experiments.

For the LLM selection, we suggest using the new reasoning models which can achieve better results than the traditional LLMs without reasoning. In our experiments, the Qwen3-series can achieve better results than the GPT-oss-20B series, although the later have more parameters. We recommend using Qwen3-series to do this task.

For the prompting methods, providing the detailed defination of the tasks and give the model generated examples, and incorpate with the manully written Chain-of-Thought can make the models achieves best performance.

Results on different categories do not follow the distribution of the datasets. Analysis from the best performing models on this task shows that LLMs' performance on the

LLMs not always follow the prompts. During our experiments, we find some of the results

6 DISCUSSION

Our results suggest that new reasoning LLMs are more suitable for fine-grained ML SATD classification than traditional non-reasoning models. In particular, the Qwen3 series consistently shows strong performance, and the best overall result is achieved by Qwen3-8B under vanilla prompting with three examples and manually written chain-of-thought. This indicates that, for this task, instruction-following and reasoning ability are more important than parameter size alone. In addition, code-oriented models do not consistently outperform general-purpose models, suggesting that the task requires not only code understanding but also the interpretation of maintenance intent and subtle semantic differences among labels.

Prompt design also plays a critical role. Vanilla and binary prompting consistently outperform hierarchical prompting, indicating that multi-stage decomposition introduces additional error propagation. Adding LLM-generated examples is generally beneficial, but more examples do not necessarily yield better results; in most cases, fewer than three examples are sufficient. Manually written chain-of-thought further improves performance, likely because it helps the model distinguish semantically overlapping labels more effectively. Therefore, an effective prompt for this task should include clear label definitions, a small number of representative examples, and lightweight reasoning guidance.

Finally, the category-level results show that classification difficulty does not simply follow label frequency. Although several high-frequency categories are recognized relatively well, the confusion matrix reveals that many errors are concentrated among semantically similar labels, especially data-, configuration-, and dependency-related types. More abstract or low-frequency categories remain difficult to classify reliably. Moreover, LLMs do not always strictly follow the required output format, which becomes more problematic in a 23-class fine-grained setting. Overall, while

open LLMs already provide a promising training-free solution for this task, semantic overlap among labels and imperfect prompt following remain important challenges for future work.

7 THREATS TO VALIDITY

In this section, we will describe the threats that can affect the validity of our study.

Internal validity. We test our results on a famous and widely used dataset due to the limited dataset available in this area. Some concerns about the generalized may rised about the results on the other datasets. We think the LLMs have a strong ability to performance well on other datasets, and these results found in our paper can generalized to other datasets and other models as well.

External validity. During our testing, we selected the open sourced LLMs due to the reasons of private. Some private code in the company are not suitable for senting to the online LLMs and these open sourced LLMs are the best option for develop in their own service and doing this task. Our results best show the performance of these models. The evouluation of these LLMs are very repaid and some new LLMs are emeraged after we finished the experiments. Further work can include these new models.

8 CONCLUSION

This paper presents the first large-scale empirical study on the effectiveness of open LLMs for fine-grained ML-SATD classification. Using the O'Brien dataset with 23 ML-specialized SATD types, we evaluated nine representative open LLMs and systematically studied the impact of prompting strategies, LLM-generated examples, reasoning mode, and their combinations. Our results show that prompt design is critical for this task: vanilla and binary prompting consistently outperform hierarchical prompting, a small number of examples is generally more effective than using many examples, and reasoning further improves performance. Among all evaluated settings, the best result is achieved by Qwen3-8B with vanilla prompting, three generated examples, and manually written chain-of-thought, reaching a weighted F1-score of 0.381.

Overall, our findings demonstrate that open LLMs provide a practical and training-free solution for fine-grained ML-SATD classification under data-scarce and highly imbalanced settings. Beyond establishing a strong baseline, this study also offers concrete guidance for model selection and prompt design in this task. At the same time, the results suggest that fine-grained ML-SATD classification remains challenging, especially because of the sparsity of many labels and the semantic overlap among debt types. Future work can extend this study by evaluating stronger closed-source and newly released models, validating the findings on additional ML-SATD datasets, and exploring more realistic settings such as multi-label classification and broader ML-specific technical debt management tasks.

REFERENCES

[1] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925* (2025).

[2] Mohannad Alahdab and Gül Çalıklı. 2019. Empirical analysis of hidden technical debt patterns in machine learning software. In *International Conference on Product-Focused Software Process Improvement*. Springer, 195–202.

[3] Francesca Arcelli Fontana, Juri Di Rocco, Davide Di Ruscio, Amleto Di Salle, and Phuong T. Nguyen. 2025. Binary and multi-class classification of Self-Admitted Technical Debt: How far can we go? *Information and Software Technology* 187 (2025), 107862.

[4] Aaditya Bhatia, Foutse Khomh, Bram Adams, and Ahmed E Hassan. 2023. An empirical study of self-admitted technical debt in machine learning software. *ACM Transactions on Software Engineering and Methodology* (2023).

[5] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D Sculley. 2017. The ML test score: A rubric for ML production readiness and technical debt reduction. In *2017 IEEE international conference on big data (big data)*. IEEE, 1123–1132.

[6] Xin Chen, Dongjin Yu, Xulin Fan, Lin Wang, and Jie Chen. 2021. Multiclass classification for self-admitted technical debt based on XGBoost. *IEEE Transactions on Reliability* 71, 3 (2021), 1309–1324.

[7] Everton da Silva Maldonado, Emad Shihab, and Nikolaos Tsantalis. 2017. Using natural language processing to automatically detect self-admitted technical debt. *IEEE Transactions on Software Engineering* 43, 11 (2017), 1044–1062.

[8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[9] Qiao Huang, Emad Shihab, Xin Xia, David Lo, and Shanping Li. 2018. Identifying self-admitted technical debt in open source projects using text mining. *Empirical Software Engineering* 23, 1 (2018), 418–451.

[10] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

[11] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.

[12] Jun Li, Lixian Li, Jin Liu, Xiao Yu, Xiao Liu, and Jacky Wai Keung. 2025. Large language model ChatGPT versus small deep learning models for self-admitted technical debt detection: Why not together? *Software: Practice and Experience* 55, 1 (2025), 3–28.

[13] Jiakun Liu, Qiao Huang, Xin Xia, Emad Shihab, David Lo, and Shanping Li. 2020. Is using deep learning frameworks free? characterizing technical debt in deep learning frameworks. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Society*. 1–10.

[14] Everton da S Maldonado and Emad Shihab. 2015. Detecting and quantifying different types of self-admitted technical debt. In *2015 IEEE 7Th international workshop on managing technical debt (MTD)*. IEEE, 9–15.

[15] Sota Nakashima, Yuta Ishimoto, Masanari Kondo, Tao Xiao, and Yasutaka Kamei. 2025. How Far Have LLMs Come Toward Automated SATD Taxonomy Construction?. In *2025 32nd Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 832–836.

[16] David OBrien, Sumon Biswas, Sayem Imtiaz, Rabe Abdalkareem, Emad Shihab, and Hridesh Rajan. 2022. 23 shades of self-admitted technical debt: an empirical study on machine learning software. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 734–746.

[17] Federica Pepe, Fiorella Zampetti, Antonio Mastropaolo, Gabriele Bavota, and Massimiliano Di Penta. 2024. A taxonomy of self-admitted technical debt in deep learning systems. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 388–399.

[18] Aniket Potdar and Emad Shihab. 2014. An exploratory study on self-admitted technical debt. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 91–100.

[19] Gilberto Recupito, Fabiano Pecorelli, Gemma Catolino, Valentina Lenarduzzi, Davide Taibi, Dario Di Nucci, and Fabio Palomba. 2024. Technical debt in ai-enabled systems: On the prevalence, severity, impact, and management strategies for code and architecture. *Journal of Systems and Software* 216 (2024), 112151.

[20] Xiaoxue Ren, Zhenchang Xing, Xin Xia, David Lo, Xinyu Wang, and John Grundy. 2019. Neural network-based detection of self-admitted technical debt: From performance to explainability. *ACM transactions on software engineering and methodology (TOSEM)* 28, 3 (2019), 1–45.

[21] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[22] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, and Michael Young. 2014. Machine learning: The high interest credit card of technical debt. In *SE4ML: software engineering for machine learning (NIPS 2014 Workshop)*, Vol. 8. Cambridge, MA.

[23] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. 2015. Hidden technical debt in machine learning systems. *Advances in neural information processing systems* 28 (2015).

[24] Mohammad Sadegh Sheikhaei, Yuan Tian, Shaowei Wang, and Bowen Xu. 2024. An empirical study on the effectiveness of large language models for satd identification and classification. *Empirical Software Engineering* 29, 6 (2024), 159.

[25] Emmanuel Iko-Ojo Simon, Chirath Hettiarachchi, Alex Potanin, Hanna Suominen, and Fatemeh Fard. 2026. Automated detection of algorithm debt in deep learning frameworks: an empirical study. *Empirical Software Engineering* 31, 3 (2026), 66.

[26] Dionysios Sklavenitis and Dimitris Kalles. 2024. Measuring technical debt in ai-based competition platforms. In *Proceedings of the 13th hellenic conference on artificial intelligence*. 1–10.

[27] CodeGemma Team, Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A Choquette-Choo, Jingyue Shen, Joe Kelley, et al. 2024. Codegemma: Open code models based on gemma. *arXiv preprint arXiv:2406.11409* (2024).

[28] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786* (2025).

[29] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118* (2024).

[30] Supatsara Wattanakriengkrai, Napat Srisermphoak, Sahawat Sintoplertchaikul, Morakot Choetkiertikul, Chaiyong Ragkhitwetsagul, Thanwadee Sunetnanta, Hideaki Hata, and Kenichi Matsumoto. 2019. Automatic classifying self-admitted technical debt using n-gram IDF. In *2019 26th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 316–322.

[31] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[32] Jiaojiao Yu, Xu Zhou, Xiao Liu, Jin Liu, Zhiwen Xie, and Kunsong Zhao. 2023. Detecting multi-type self-admitted technical debt with generative adversarial network-based neural networks. *Information and Software Technology* 158 (2023), 107190.

Table 9. Results of layered hybrid setting

Type	Qwen3-14B			gemma-2-9b			gpt-oss-20b		
	precision	recall	f1	precision	recall	f1	precision	recall	f1
AWR	**0.643**	0.422	0.509	0.338	**0.797**	0.474	0.529	0.578	**0.552**
RDB	0.062	**0.625**	0.112	0.103	0.375	0.162	**0.167**	0.500	**0.250**
DCE	0.182	**1.000**	0.308	**0.500**	**1.000**	**0.667**	0.167	**1.000**	0.286
CFO	0.187	**0.452**	0.264	**0.245**	0.387	**0.300**	0.184	**0.452**	0.262
CDD	0.509	**0.358**	**0.420**	0.632	0.296	0.403	**0.719**	0.284	0.407
DTD	0.718	0.204	0.318	**0.773**	0.248	0.376	0.656	**0.431**	**0.520**
MDL	0.212	**0.275**	0.239	0.191	0.225	0.207	**0.312**	0.250	**0.278**
PRF	0.250	0.333	0.286	**0.385**	**0.556**	**0.455**	0.060	0.333	0.102
SCL	0.250	**0.286**	**0.267**	0.222	0.190	0.205	**0.286**	0.095	0.143
weighted avg	0.517	0.318	0.350	0.519	0.366	0.364	**0.530**	**0.392**	**0.420**

Table 10. Definition of the father categories

Dim. Name	Definition
Awareness (AWR)	Lack of developer understanding or certainty about ML algorithms, APIs, data, or model behavior. Often arises due to "black box" nature of ML models.
Readability (RDB)	Model code has poor legibility or unclear logic, especially in ML contexts (e.g., poorly named variables, dense math code).
Duplicate Code Elimination (DCE)	Identical or near-identical ML model or preprocessing logic appears in multiple places.
Configurable Options (CFO)	Missing or poorly implemented configuration points for weights, network layers, or model parameters.
Code Dependency (CDD)	Code depends on other code or libraries in a way that blocks maintenance, requires workarounds, or is outdated.
Data Dependency (DTD)	Issues in how data is processed, represented, or stored that hinder maintainability or correctness.
Modularity (MDL)	Poor modular boundaries or design make the system harder to evolve, reuse, or maintain.
Performance (PRF)	Performance or robustness is insufficient, requiring evaluation, optimization, or reliability improvement.
Scalability (SCL)	Prototype-level code is used in production without scaling optimizations.